

PROJECT REPORT

Project Name	Implement Client Script & UI Policy (Incident)
Team ID	SWTID-2026-3114
Platform	ServiceNow – Incident Management
Submission Date	3rd October 2026

1. INTRODUCTION

1.1 Project Overview

The project “**Implement Client Script & UI Policy (Incident)**” focuses on improving the user experience and data accuracy of the ServiceNow Incident Management module. The solution uses **Client Scripts and UI Policies** to dynamically control Incident fields based on user actions and Incident conditions.

The implementation includes automatic updating of the Urgency field when Impact is set to High, making Urgency read-only for High Impact Incidents, validating the Assigned To field before saving, and preventing direct State changes through list editing.

These controls operate on the Incident form and provide immediate feedback to users without requiring unnecessary form submissions.

1.2 Purpose

The main purpose of this project is to reduce incorrect and incomplete Incident data entry by implementing dynamic field controls and validations in ServiceNow.

The project aims to:

- Automatically set Urgency to High for High Impact Incidents.
 - Make the Urgency field read-only when Impact is High.
 - Ensure Assigned To is provided before saving a High Impact Incident.
 - Prevent State changes through direct list editing.
 - Provide clear messages and feedback to users.
 - Improve the consistency and accuracy of Incident records.
-

2. IDEATION PHASE

2.1 Problem Statement

ServiceNow Incident users may enter incomplete or inconsistent information because important fields are not dynamically controlled and validated based on Incident conditions.

High Impact Incidents may require specific field behavior, such as setting Urgency to High and ensuring that the Incident is assigned to a user. Without appropriate controls, users may also make unintended State changes through list editing.

Therefore, a solution is required to dynamically control Incident fields, validate required information, and prevent incorrect updates.

2.2 Empathy Map Canvas

User: ServiceNow Incident Management User

Think & Feel

- Wants to create accurate Incident records.
- Wants clear guidance while entering information.
- Wants to avoid incorrect Incident updates.
- Wants the system to automatically handle dependent fields.

Hear

- System messages about required information.
- Validation messages when information is missing.
- Instructions to update Incident information correctly.

See

- Incident form and Incident list.
- Impact, Urgency, Assigned To and State fields.
- Informational and error messages.

Say & Do

- Creates and updates Incidents.
- Changes Impact and other Incident fields.
- Assigns Incidents to users.
- Saves Incident records.
- Updates Incident State.

Pain

- Incomplete Incident information.
- Manual updating of dependent fields.
- Missing Assigned To information.
- Accidental State changes.

Gain

- Automatic field updates.
- Clear validation.
- Controlled field behavior.
- More accurate Incident records.
- Consistent Incident management.

2.3 Brainstorming

The team identified the following possible solutions:

1. Automatically set Urgency to High when Impact is High.
2. Make Urgency read-only for High Impact Incidents.
3. Validate Assigned To before saving a High Impact Incident.
4. Prevent State changes through direct list editing.
5. Display clear informational and error messages.
6. Dynamically control Incident fields based on conditions.

The ideas were grouped into **Automation, Field Control, Validation, and Restriction**. The selected solutions were prioritized based on their relevance to the identified Incident management problems and their feasibility within ServiceNow.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The Incident user's journey consists of the following stages:

Create Incident → Enter Details → Set Impact → Automatic Urgency Control → Assign Incident → Validate → Save → Manage State

Stage	User Action	System Response	User Need
Create Incident	Opens Incident form	Displays Incident fields	Easy form access
Enter Details	Enters Incident information	Accepts user input	Accurate data entry

Set Impact	Selects High Impact	Triggers UI Policy and Client Script	Automatic field control
Control Urgency	Reviews Urgency	Sets Urgency to High and makes it read-only	Correct Urgency
Assign Incident	Selects Assigned To	Stores assignment information	Complete assignment
Validate	Attempts to save	Checks Assigned To for High Impact	Prevent incomplete records
Manage State	Attempts State update	Blocks direct list editing	Prevent unintended changes
Complete Update	Saves valid Incident	Stores Incident record	Consistent Incident information

3.2 Solution Requirement

Functional Requirements

FR No.	Functional Requirement	Sub Requirement
FR-1	High Impact Field Control	Trigger UI Policy when Impact is High
FR-2	Automatic Urgency Management	Set Urgency to High when Impact is High; make Urgency read-only
FR-3	Incident Validation	Validate Assigned To before saving High Impact Incidents

FR-4	State Change Restriction	Prevent State changes through list editing
FR-5	Condition Reversal	Reverse UI Policy changes when Impact is no longer High
FR-6	Incident Update	Allow valid Incident records to be saved successfully

Non-functional Requirements

NFR No.	Requirement	Description
NFR-1	Usability	Provide clear field behavior and user feedback.
NFR-2	Security	Prevent unintended State changes through list editing.
NFR-3	Reliability	Apply Client Scripts and UI Policies consistently.
NFR-4	Performance	Execute field updates and validations with immediate response.
NFR-5	Availability	Make the configured controls available whenever Incidents are accessed.
NFR-6	Maintainability	Keep the configuration clear and easy for administrators to modify.

3.3 Data Flow Diagram

The data flow of the solution is:

Incident User

↓

ServiceNow Incident Form / List View

↓

Client Scripts & UI Policies

↓

Validation and Field Control

↓

Incident Record

Main Data Flows

- **Impact = High** → Client Script → Urgency set to High.
- **Impact = High** → UI Policy → Urgency becomes read-only.
- **High Impact + Assigned To empty** → onSubmit Client Script → Error message → Save prevented.
- **State edited in List** → onCellEdit Client Script → Warning message → Update blocked.
- **Valid Incident information** → Successful Save → Incident Record.

3.4 Technology Stack

Component	Technology Used
Platform	ServiceNow
Module	Incident Management
User Interface	ServiceNow Incident Form and List View
Scripting Language	JavaScript
Client-side API	GlideForm (<code>g_form</code>)
Configuration	UI Policy and UI Policy Actions
Client Script – 1	onChange
Client Script – 2	onSubmit
Client Script – 3	onCellEdit

Data Storage	ServiceNow Incident Database
Deployment	ServiceNow Cloud Platform

4. PROJECT DESIGN

4.1 Problem Solution Fit

The project addresses the problem of incorrect and incomplete Incident information by implementing dynamic controls and validations.

Problem	Solution
Urgency may not match High Impact	Automatically set Urgency to High
Urgency may be changed incorrectly	Make Urgency read-only
Assigned To may be missing	Validate Assigned To before saving
State may be changed accidentally through list editing	Block State list editing
Users may need immediate guidance	Display information and error messages

The proposed solution directly addresses the identified Incident management problems using native ServiceNow Client Scripts and UI Policies.

4.2 Proposed Solution

The proposed solution implements a set of Client Scripts and a UI Policy on the ServiceNow Incident table.

Main Features

- **High Impact Control:** UI Policy activates when Impact is High.
- **Automatic Urgency:** onChange Client Script sets Urgency to High.
- **Urgency Control:** UI Policy makes Urgency read-only.
- **Assigned To Validation:** onSubmit Client Script prevents saving when Assigned To is empty for High Impact Incidents.
- **State Restriction:** onCellEdit Client Script prevents direct State changes from the Incident list.
- **User Feedback:** Information and error messages are displayed during relevant actions.
- **Reverse Condition:** UI Policy changes are reversed when the Impact condition is no longer true.

4.3 Solution Architecture

The solution follows this architecture:

Incident User

↓

ServiceNow Incident Form / List View

↓

Client Scripts + UI Policies

↓

Validation & Field Control

↓

ServiceNow Incident Record

Architecture Components

- **Incident User:** Creates and updates Incident records.
 - **Incident Form:** Provides the main user interface.
 - **UI Policy:** Controls fields based on Impact conditions.
 - **onChange Client Script:** Updates Urgency automatically.
 - **onSubmit Client Script:** Validates Assigned To.
 - **onCellEdit Client Script:** Prevents State list editing.
 - **Incident Database:** Stores the final Incident record.
-

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was planned and executed through the following activities:

Phase	Activity	Outcome
Ideation	Problem identification	Identified Incident data-entry and field-control problems
Ideation	Empathy mapping	Identified user needs and pain points
Ideation	Brainstorming	Identified possible ServiceNow solutions
Requirement Analysis	Customer Journey	Mapped the Incident user journey
Requirement Analysis	Requirements	Defined functional and non-functional requirements
Requirement Analysis	DFD	Defined system and data flow
Requirement Analysis	Technology Stack	Identified ServiceNow technologies
Project Design	Problem-Solution Fit	Matched identified problems with solutions
Project Design	Proposed Solution	Defined Client Script and UI Policy implementation
Project Design	Architecture	Designed the solution flow
Implementation	UI Policy	Created High Impact Control

Implementation	Client Scripts	Implemented onChange, onSubmit and onCellEdit scripts
Testing	Functional Testing	Verified field behavior and validations
Documentation	Final Report	Documented implementation and results

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The solution was tested on the ServiceNow Incident form to verify that Client Scripts and UI Policies respond immediately to user actions.

Test Scenario	Expected Result	Observed Result
Change Impact to High	Urgency automatically changes to High	Passed
High Impact Incident	Urgency becomes read-only	Passed
High Impact + Assigned To empty + Save	Save is prevented and error is displayed	Passed
High Impact + Assigned To populated + Save	Incident saves successfully	Passed
Change Impact from High to another value	UI Policy changes are reversed	Passed

Edit State directly from list	State update is blocked and warning displayed	Passed
Update Incident through form	Valid changes are saved successfully	Passed

The testing confirmed that the configured Client Scripts and UI Policy execute on the relevant user actions without requiring unnecessary page refreshes.

7. RESULTS

7.1 Output Screenshots

The implemented solution was successfully tested in the ServiceNow Incident module.

The screenshots for the following details has been added to the project repository.

High Impact Control

Description: Shows the High Impact condition and the corresponding field behavior.

Automatic Urgency Update

Description: Shows Urgency being automatically set to High when Impact is changed to High.

Urgency Read-only

Description: Shows the Urgency field becoming read-only for a High Impact Incident.

Assigned To Validation

Description: Shows the validation message when a High Impact Incident is submitted without Assigned To.

Successful Save

Description: Shows a High Impact Incident being successfully saved after providing the required assignment information.

State List Edit Blocking

Description: Shows the warning displayed when a user attempts to change State directly from the Incident list.

Form-based Update

Description: Shows a valid Incident update being performed through the Incident form.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Improves Incident data accuracy.
- Reduces incorrect manual data entry.
- Automatically updates dependent fields.
- Provides immediate validation and user feedback.
- Prevents unintended State changes through list editing.
- Improves consistency of High Impact Incident handling.
- Uses native ServiceNow configuration and scripting features.

Disadvantages

- The Client Scripts and UI Policies require proper configuration and maintenance.
 - Changes to Incident field names or values may require script updates.
 - The solution is specific to the configured Incident conditions.
 - Excessive Client Scripts and UI Policies may make future configuration management more complex.
-

9. CONCLUSION

The **Implement Client Script & UI Policy (Incident)** project successfully demonstrates how ServiceNow Client Scripts and UI Policies can be used to improve Incident management.

The implemented solution automatically controls Urgency for High Impact Incidents, validates Assigned To information before saving, makes Urgency read-only when required, and prevents direct State changes through list editing.

The testing confirmed that the configured controls provide immediate responses to user actions and help maintain more accurate and consistent Incident records.

10. FUTURE SCOPE

The solution can be enhanced in the future with additional ServiceNow automation and validation features, such as:

- Extending dynamic field control to additional Incident fields.
 - Adding more condition-based UI Policies.
 - Introducing role-based field behavior where required.
 - Adding automated notifications for important Incident conditions.
 - Extending similar validation rules to other ServiceNow tables.
 - Integrating additional workflow or Flow Designer automation for Incident management.
 - Adding more comprehensive automated testing for different Incident scenarios.
-

11. APPENDIX

Source Code

The project uses JavaScript Client Scripts in ServiceNow.

1. Auto Set Urgency for High Impact

```

function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == "") {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage(
            'Urgency set to High for High impact incident.'
        );
    }
}

```

2. Prevent Save if Assigned To is Missing

```

function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == "") {

        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }

    return true;
}

```

3. Prevent State Change via List Edit

```

function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert(
        'State cannot be updated using list editing. Please open the Incident.'
    );

    callback(false);
}

```

Dataset Link

Not Applicable – This project does not use an external dataset. The implementation uses ServiceNow Incident records and configuration data.

GitHub & Project Demo Link

GitHub Repository:

<https://github.com/PAVITHRA-20070120/Implement-Client-Script-UI-Policy-Incident-.git>